

Лабораторная работа 1: "Решение нелинейных уравнений и систем"

Задание

Дана функция f .

- Определить количество корней уравнения $f(x) = 0$. и отделить каждый корень. При этом разрешается использовать графический способ, однако необходимо доказать, что других корней уравнение не имеет;
- Вычислить все корни с точностью $\varepsilon = 10^{-12}$ методом бисекции и методом, соответствующим вашему варианту (критерий останова - $|f(x_k)| < \varepsilon$);
- Экспериментально сравнить скорость сходимости двух использованных методов. Построить совмещенные диаграммы сходимости для каждого корня. Сделать выводы.

Отчёт о лабораторной работе должен включать:

- полное описание процесса отделения корней: графики, интервалы отделимости для каждого корня, доказательство того, что других корней нет (можно от руки на бумаге);
- диаграммы сходимости для каждого корня;
- комментарии по результатам эксперимента;
- исходный код программы.

N 1.1.20

$$f(x) = \left(\frac{1}{16}\right)^x - \log_{\frac{1}{16}}(x)$$

Метод Ньютона с постоянной производной.

N 2.10

$$x_{i-1} - 2x_i + x_{i+1} = h^2 g(x_i, \frac{x_{i+1} - x_{i-1}}{2h}), \quad i = 1, \dots, n-1$$
$$g(u, v) = 2 - uv, \quad h = \frac{2\pi}{n}, \quad x_0 = 10, \quad x_n = 0.$$

Написать программу, которая решает данную систему методом Ньютона для произвольного значения n .

В качестве начального приближения использовать значения $x_i^0 = 10(1 - \frac{i}{n})$, $x = \overline{1, n-1}$
Для решения СЛАУ использовать метод прогонки. Критерий останова итерационного процесса: $\|f(x^k)\| < 10^{-10}$.

Провести вычислительный эксперимент для $n = 10, 100, 1000$. Для каждого значения n

построить диаграммы сходимости, а также точечные графики решения (множество точек (i, x_i)).

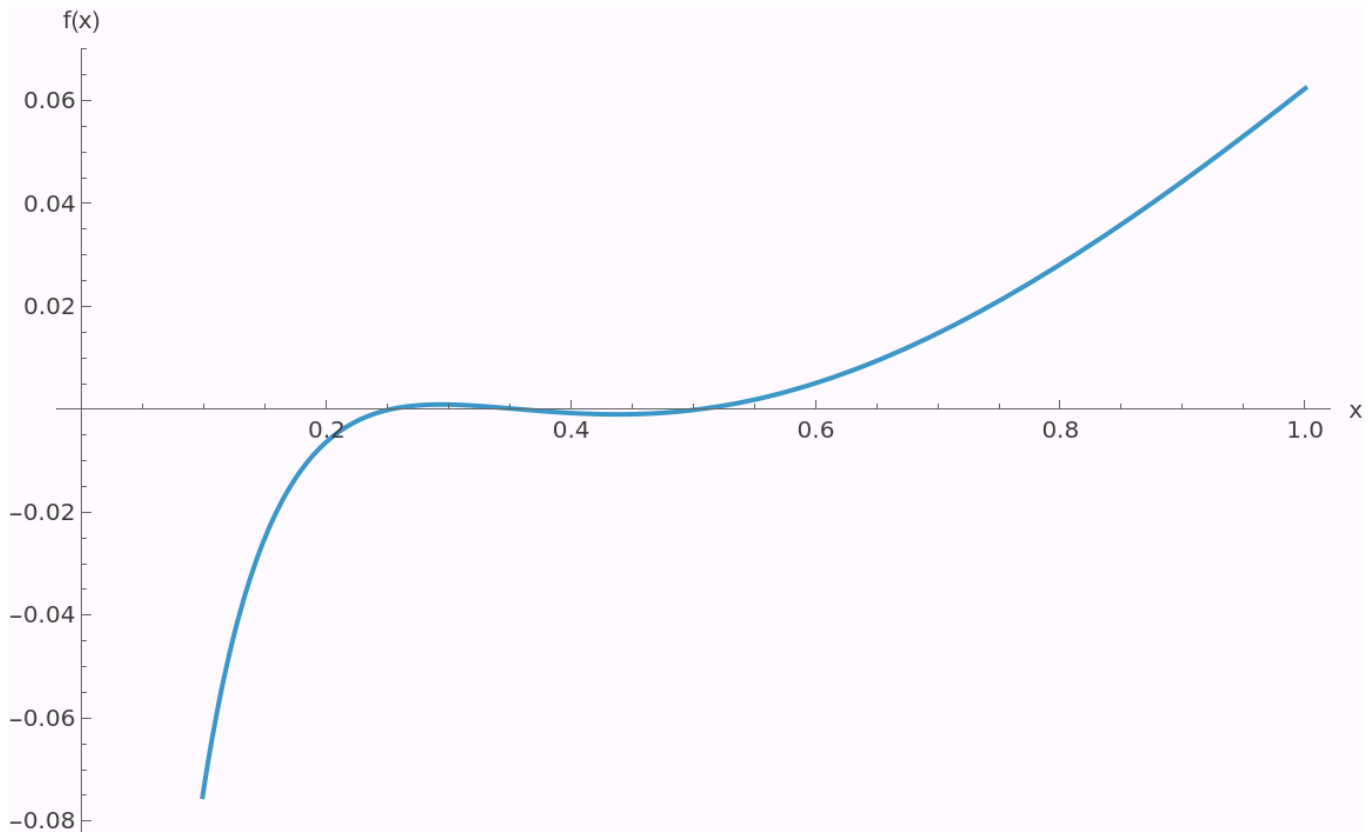
Решение

Задача 1

Определяем количество корней

Заметим, что $x > 0$, т. к. логарифм определен только на положительных числах. При помощи Wolfram Mathematica построим график $f(x)$:

```
Plot[(1/16)^x - Log[1/16, x], {x, 0, 1}, ImageSize -> Large, AxesLabel -> {"x", "f(x)"}]
```



Можем видеть, что в интервале $(0, 1)$ уравнение $f(x) = 0$ имеет 3 корня. Покажем, что других корней нет.

$$f(x) = \left(\frac{1}{16}\right)^x - \log_{\frac{1}{16}}(x) = 16^{-x} + \log_{16} x$$

Первое слагаемое $16^{-x} > 0$, а второе $-\log_{16} x \geq 0$, $x \geq 1$, значит $f(x) > 0$, $x \geq 1$ - всего 3 корня.

Отделяем корни.

Попробуем степени 2 в качестве корней:

$$f\left(\frac{1}{2}\right) = \frac{1}{4} + \left(-\frac{1}{4}\right) = 0, \quad f\left(\frac{1}{4}\right) = \frac{1}{2} + \left(-\frac{1}{2}\right) = 0$$

Третий корень лежит где-то между ними. Можем взять интервалы:

$$[a_1, b_1] = [0.2, 0.3], \quad [a_2, b_2] = [0.3, 0.45], \quad [a_3, b_3] = [0.45, 0.55]$$

Код

Реализовали методы бисекции и Ньютона с постоянной производной. Для исследования взяли несимметричные интервалы для известных корней, чтобы метод бисекции не сошелся за одну итерацию.

```
#include <cassert>
#include <cmath>
#include <cstdint>
#include <cstdlib>
#include <fstream>
#include <functional>
#include <iomanip>
#include <iostream>
#include <ostream>
#include <utility>
#include <vector>

static constexpr double kEps = 1.e-12;
static constexpr size_t kIters = 1e3;

const double kRevLog16 = 1. / std::log(16.);
const double kLog16 = std::log(16.);

double F(double x) { return std::pow(16., -x) + std::log(x) * kRevLog16; }
double Df(double x) { return -std::pow(16., -x) * kLog16 + 1. / x *
kRevLog16; }

const std::vector<std::pair<double, double>> kIntervals = {
    {.15, .3}, {.3, .45}, {.45, .6}};

template <typename T>
void MyPrint(const std::vector<T>& vec, std::ostream& out = std::cerr) {
    out << (vec.empty() ? -1 : static_cast<int>(vec.size())) << "\n";
    for (const T& elem : vec) {
        out << elem << " ";
    }
    out << "\n";
}
```

```

}

double BisectionMethod(double a, double b, std::vector<double>& pts) {
    double x = a;
    double fx = F(a);

    size_t cnt{};
    while (std::abs(fx) > kEps) {
        x = (a + b) / 2;
        fx = F(x);

        pts.push_back(std::abs(fx));
        if (fx * F(a) < 0) {
            b = x;
        } else {
            a = x;
        }

        if (cnt++ > kIters || x < 0) {
            std::cerr << "[" << a << ", " << b << "] is broken\n";
            pts.clear();
            break;
        }
    }
    return x;
}

double NewtonConstDfMethod(double a, double b, std::vector<double>& pts) {
    const double kRevMu = 1. / Df(a);
    std::cerr << "mu=" << 1. / std::abs(kRevMu) << "\n";

    double x = a;
    double fx = F(a);

    size_t cnt{};
    while (std::abs(fx) > kEps) {
        x -= fx * kRevMu;

        fx = F(x);
        pts.push_back(std::abs(fx));
        if (cnt++ > kIters || x < 0) {
            std::cerr << "x = " << x << ", [" << a << ", " << b
                << "] is broken\n";
            pts.clear();
            break;
        }
    }
}

```

```

    }
    return x;
}

void ProcMethod(
    std::function<double(double, double, std::vector<double>&)> method,
    std::ostream& out) {
    out << std::setprecision(12);

    std::vector<double> pts;
    for (auto [a, b] : kIntervals) {
        out << method(a, b, pts) << "\n";
        MyPrint(pts, out);
        pts.clear();
    }
}

int main() {
    std::ofstream foutb("outputB.txt");
    std::ofstream foutn("outputN.txt");

    ProcMethod(BisectionMethod, foutb);
    ProcMethod(NewtonConstDfMethod, foutn);

    return 0;
}

```

Код для рисования графиков:

```

import matplotlib.pyplot as plt

def read_results(filename):
    residuals = []

    with open(filename, "r") as f:
        lines = f.read().splitlines()

    for i in range(0, len(lines), 3):
        if i + 1 >= len(lines):
            break
        count = int(lines[i + 1].strip())
        res = list(map(float, lines[i + 2].split())) if count > 0 else []
        residuals.append(res)
    return residuals

```

```

def main():
    res_b = read_results("outputB.txt")
    res_n = read_results("outputN.txt")
    intervals = ["[0.15, 0.3]", "[0.3, 0.45]", "[0.45, 0.6]"]

    for i in range(3):
        plt.figure(figsize=(6, 4.5))

        if i < len(res_b) and res_b[i]:
            plt.plot(range(1, len(res_b[i]) + 1), res_b[i], "o-",
color="navy", label="Бисекция")

        if i < len(res_n) and res_n[i]:
            plt.plot(range(1, len(res_n[i]) + 1), res_n[i], "s-",
color="crimson", label="Ньютон")

        plt.yscale("log")
        plt.xlabel("Номер итерации k")
        plt.ylabel("Невязка |f(x_k)|")
        plt.title(f"Интервал {intervals[i]}")
        plt.grid(True, which="both", ls="--", alpha=0.5)
        plt.legend()

        filename = f"plot{i + 1}.png"
        plt.tight_layout()
        plt.savefig(filename, dpi=150)

if __name__ == "__main__":
    main()

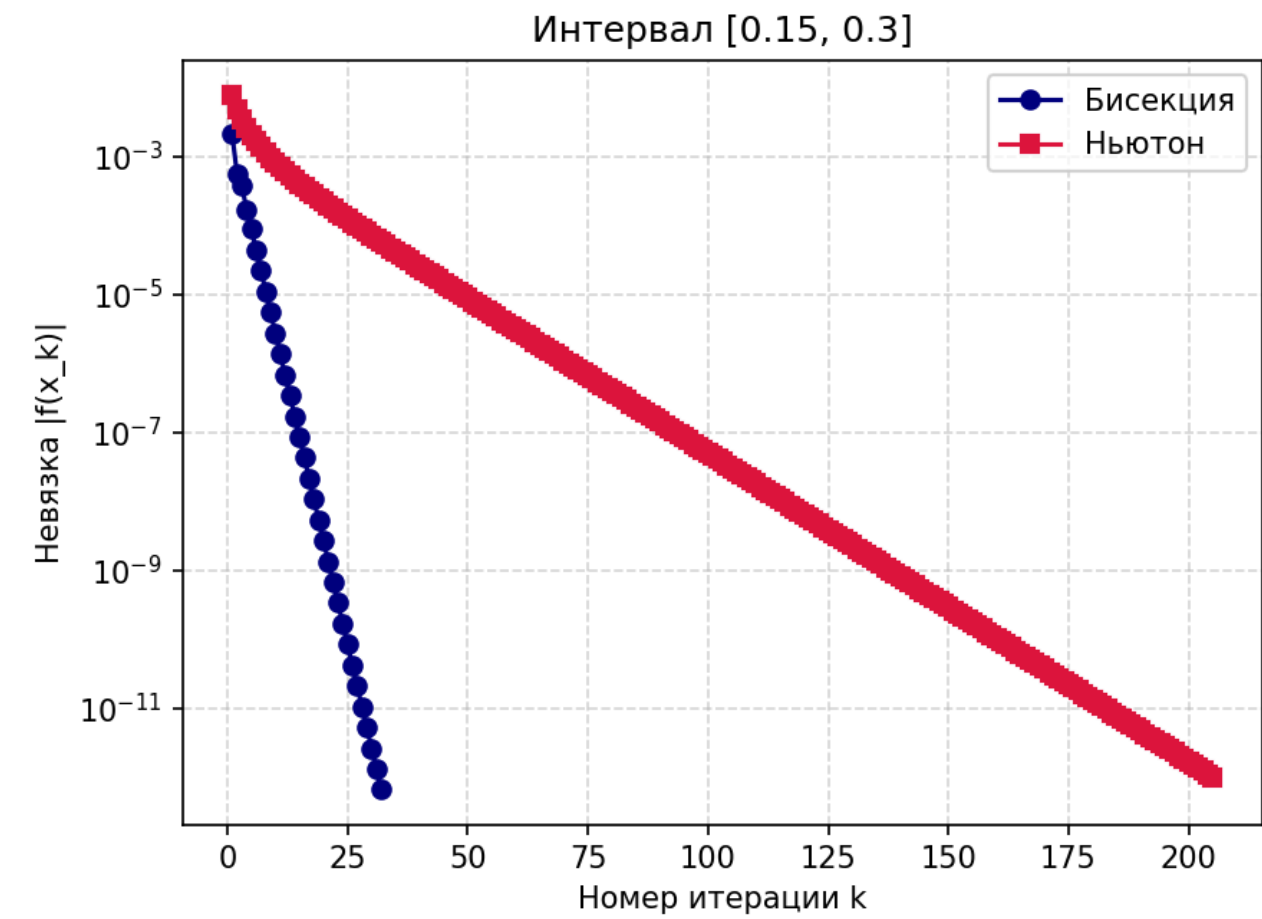
```

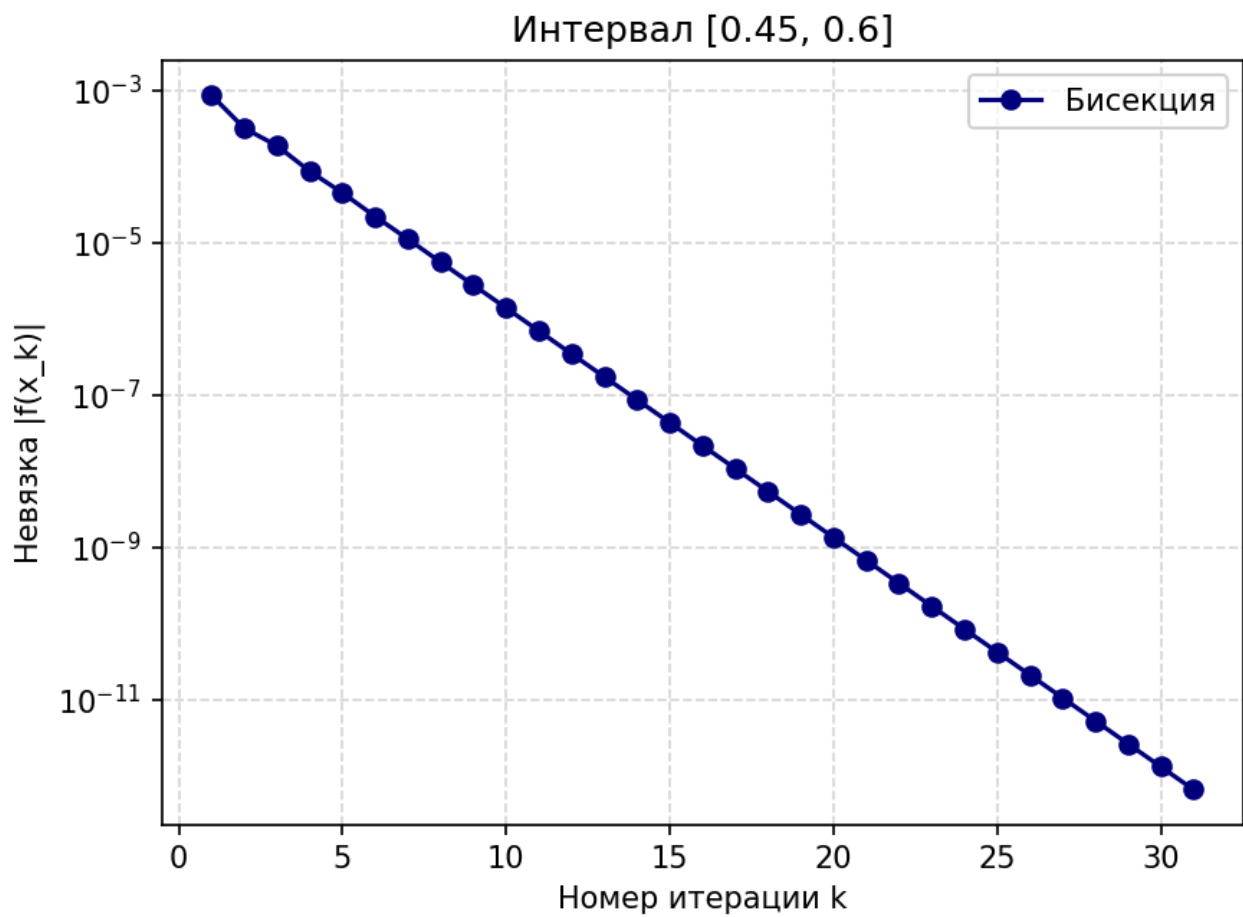
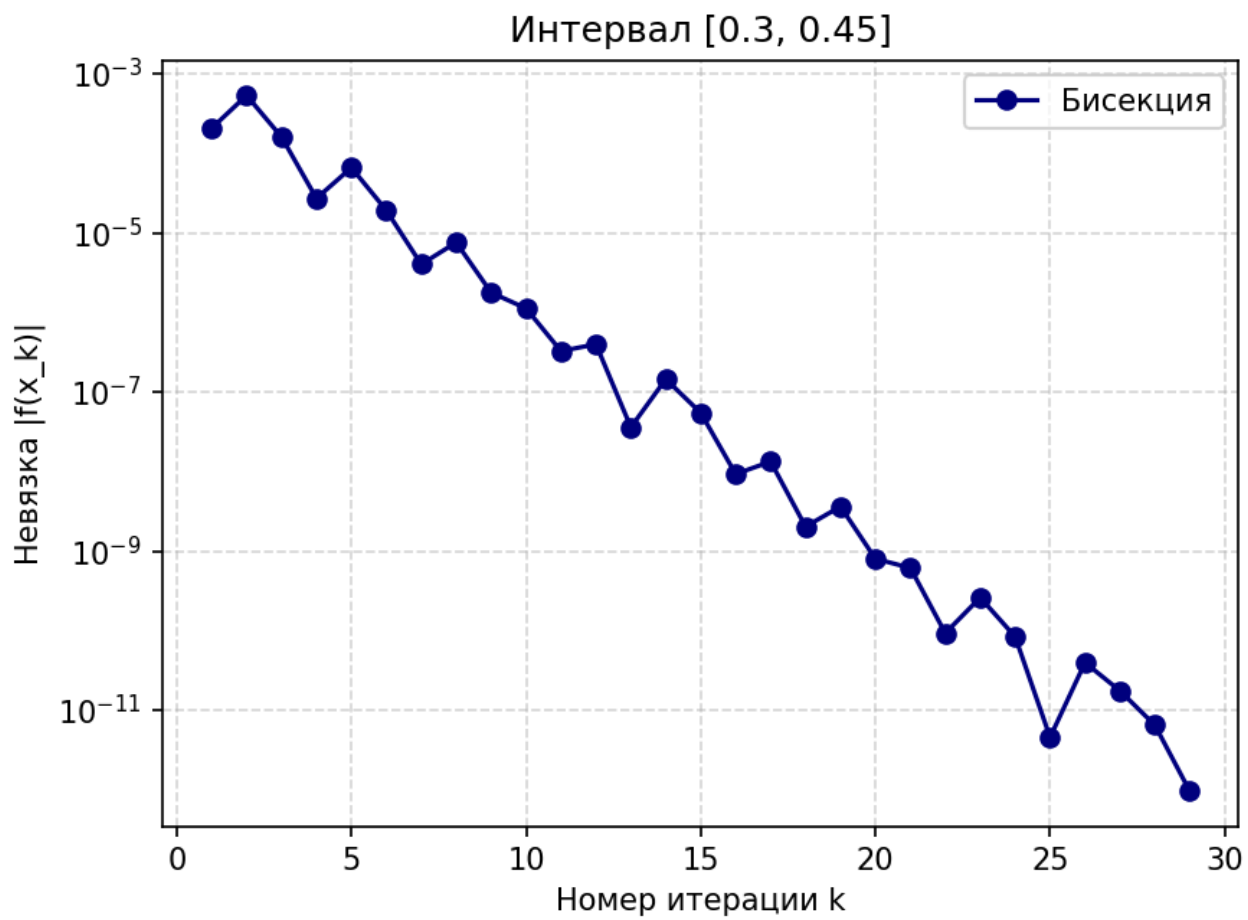
Диаграммы сходимости

Получили корни для соответствующих интервалов:

$$x_1 = 0.25, x_2 = 0.36424988983, x_3 = 0.5$$

Диаграммы сходимости:





Метод Ньютона с постоянной производной разошелся для данной функции для двух корней.

Причина следующая: на левых концах второго и третьего интервала, которые мы брали за x_0 модуль производной $f'(x_0)$ близок к нулю: 0.00459347 и 0.00528022 соответственно. Для метода

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_0)}$$

и второго интервала первое приближение получилось

$x_1 = 0.3 - 0.00103388 \cdot (-0.00459347) = 0.525077$ - вышли за пределы рассматриваемого интервала. То же самое случилось и с третьим интервалом.

На первом интервале можно видеть, что метод имеет линейный порядок сходимости.

Метод бисекции сошелся на всех интервалах, графики подтверждают его линейный порядок сходимости.

Задача 2

Выведем формулы

Имеем систему уравнений

$$x_{i-1} - 2x_i + x_{i+1} = h^2 g\left(x_i, \frac{x_{i+1} - x_{i-1}}{2h}\right), \quad i = 1, \dots, n-1$$

$$g(u, v) = 2 - uv, \quad h = \frac{2\pi}{n}, \quad x_0 = 10, \quad x_n = 0, \quad x_i^0 = 10 \left(1 - \frac{i}{n}\right), \quad x = \overline{1, n-1}$$

Чтобы получить вид $F(x) = 0$ перепишем уравнения:

$$f_i(x) = x_{i-1} - 2x_i + x_{i+1} - 2h^2 + \frac{h}{2}x_i(x_{i+1} - x_{i-1}) = 0, \quad i = \overline{1, n-1}$$

Метод Ньютона:

1. найти Δx^k из

$$f'(x^k)\Delta x^k = -f(x^k)$$

2. $x^{k+1} = x^k + \Delta x^k$

Так как f_i зависит только от трех переменных, то матрица Якоби $f'(x)$ будет трехдиагональной, для решения СЛАУ применим метод прогонки:

2.1.7. Метод прогонки

Рассмотрим СЛАУ

$$\begin{bmatrix} d_1 & e_1 & & & & \\ c_2 & d_2 & e_2 & & & \\ & c_3 & d_3 & e_3 & & \\ & & \ddots & \ddots & \ddots & \\ & & & c_{n-1} & d_{n-1} & e_{n-1} \\ & & & & c_n & d_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \\ b_n \end{bmatrix}. \quad (2.10)$$

Матрицы такой структуры называются *трехдиагональными*. В приложениях достаточно часто встречаются такие системы. Особая структура этой матрицы позволяет найти решение системы методом Гаусса за $O(n)$ операций. Получаемый метод называется *методом прогонки*.

Алгоритм метода прогонки

- 1: **for** $k = \overline{2, n}$ **do** // Прямой ход
- 2: $d_k \leftarrow d_k - e_{k-1}c_k/d_{k-1}$
- 3: $b_k \leftarrow b_k - b_{k-1}c_k/d_{k-1}$
- 4: **end for**
- 5: $x_n = b_n/d_n$
- 6: **for** $k = \overline{n-1, 1}$ **do** // Обратный ход
- 7: $x_k \leftarrow (b_k - e_k x_{k+1})/d_k$
- 8: **end for**

При этом наши диагонали будут:

$$\begin{cases} c_i = \frac{\partial f_i}{\partial x_{i-1}} = 1 - \frac{h}{2} x_i \\ d_i = \frac{\partial f_i}{\partial x_i} = -2 + \frac{h}{2} (x_{i+1} - x_{i-1}) \\ e_i = \frac{\partial f_i}{\partial x_{i+1}} = 1 + \frac{h}{2} x_i \\ b_i = -f_i(x^k) \end{cases}$$

Код

Реализовали все вышеописанные формулы на C++:

```
#include <algorithm>
#include <cmath>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

static constexpr double kEps = 1e-10;
```

```

static constexpr double kPi = 3.14159265358979;

std::vector<double> x, c, d, e, f, dx, res;

void Progonka(int n) {
    const int kM = n - 1;

    for (int k = 2; k <= kM; ++k) {
        double factor = c[k] / d[k - 1];
        d[k] = d[k] - e[k - 1] * factor;
        f[k] = f[k] - f[k - 1] * factor;
    }

    dx[kM] = f[kM] / d[kM];
    for (int k = kM - 1; k >= 1; --k) {
        dx[k] = (f[k] - e[k] * dx[k + 1]) / d[k];
    }

    dx[0] = 0.;
    dx[n] = 0.;
}

void Newton(int n) {
    const double kH = 2. * kPi / n;

    x.resize(n + 1);
    c.resize(n + 1);
    d.resize(n + 1);
    e.resize(n + 1);
    f.resize(n + 1);
    dx.resize(n + 1, 0.);

    res.clear();

    for (int i = 0; i <= n; ++i) {
        x[i] = 10. * (1. - i * 1. / n);
    }

    while (true) {
        double max_res = 0.0;

        for (int i = 1; i < n; ++i) {
            double fi = x[i - 1] - 2 * x[i] + x[i + 1] - 2 * kH * kH +
                (kH / 2.) * x[i] * (x[i + 1] - x[i - 1]);

            max_res = std::max(max_res, std::abs(fi));
        }
    }
}

```

```

        f[i] = -fi;
        c[i] = 1. - (kH / 2.) * x[i];
        d[i] = -2. + (kH / 2.) * (x[i + 1] - x[i - 1]);
        e[i] = 1. + (kH / 2.) * x[i];
    }

    res.push_back(max_res);

    if (max_res < kEps) {
        break;
    }

    Progonka(n);

    for (int i = 1; i < n; ++i) {
        x[i] += dx[i];
    }
}

std::ofstream fout_conv("conv" + std::to_string(n) + ".txt");
fout_conv << std::setprecision(10);
for (double r : res) {
    fout_conv << r << "\n";
}

std::ofstream fout_sol("sol" + std::to_string(n) + ".txt");
fout_sol << std::setprecision(10);
for (int i = 0; i <= n; ++i) {
    fout_sol << i << " " << x[i] << "\n";
}
}

int main() {
    Newton(10);
    Newton(100);
    Newton(1000);
    return 0;
}

```

После запуска получим по 2 файла `conv{i}.txt` и `sol{i}.txt`, $i=n$ для каждого n с невязками и решениями соответственно, построим требуемые графики:

```
import matplotlib.pyplot as plt
```

```

def plot_convergence():
    ns = [10, 100, 1000]
    colors = ["navy", "crimson", "forestgreen"]
    markers = ["o", "s", "^"]

    plt.figure(figsize=(7, 5))

    for n, color, marker in zip(ns, colors, markers):
        filename = f"conv{n}.txt"

        with open(filename, "r") as f:
            res = [float(line.strip()) for line in f if line.strip()]

        k = range(1, len(res) + 1)
        plt.plot(
            k,
            res,
            marker=marker,
            color=color,
            linestyle="--",
            linewidth=1.5,
            markersize=6,
            label=f"n = {n}",
        )

    plt.yscale("log")
    plt.xlabel("k")
    plt.ylabel(" $\|F(x^k)\|$ ")
    plt.title("Диаграмма сходимости метода Ньютона")
    plt.grid(True, which="both", ls="--", alpha=0.5)
    plt.legend()
    plt.tight_layout()
    plt.savefig("task2_convergence.png", dpi=300)
    plt.close()

def plot_solutions():
    ns = [10, 100, 1000]
    colors = ["navy", "crimson", "forestgreen"]

    for n, color in zip(ns, colors):
        filename = f"sol{n}.txt"

        with open(filename, "r") as f:
            lines = [line.split() for line in f if line.strip()]

```

```

        i_vals = [int(line[0]) for line in lines]
        x_vals = [float(line[1]) for line in lines]

plt.figure(figsize=(6, 4.5))

if n == 10:
    plt.plot(
        i_vals,
        x_vals,
        marker="o",
        color=color,
        linestyle="--",
        markersize=6,
        linewidth=1.5,
        label=f"n = {n}",
    )
elif n == 100:
    plt.plot(
        i_vals,
        x_vals,
        marker=".",
        color=color,
        linestyle="--",
        markersize=4,
        linewidth=1.5,
        label=f"n = {n}",
    )
else:
    plt.plot(i_vals, x_vals, color=color, linestyle="--",
linewidth=2, label=f"n = {n}")

plt.xlabel("i")
plt.ylabel("$x_i$")
plt.title(f"Решение системы (n = {n})")
plt.grid(True, ls="--", alpha=0.5)
plt.legend()

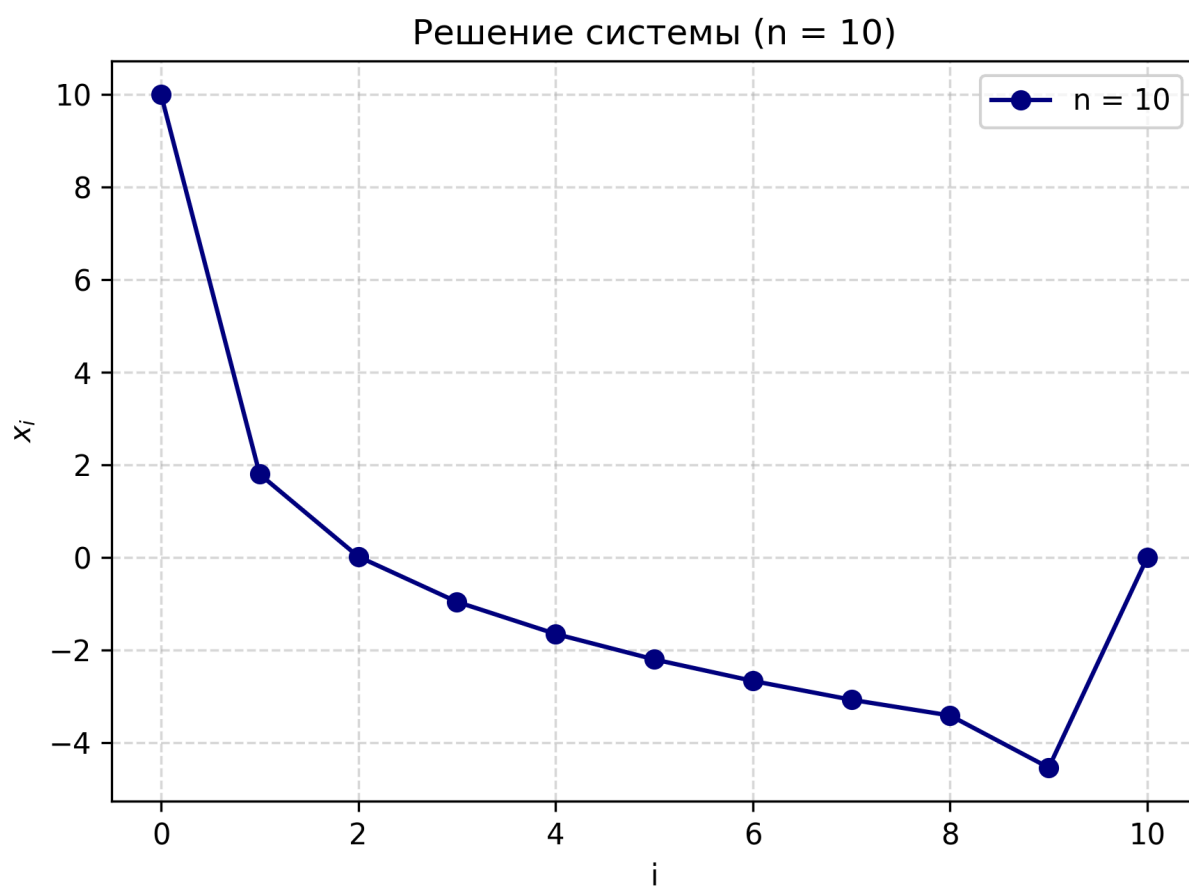
out_filename = f"task2_solution_{n}.png"
plt.tight_layout()
plt.savefig(out_filename, dpi=300)

if __name__ == "__main__":
    plot_convergence()
    plot_solutions()

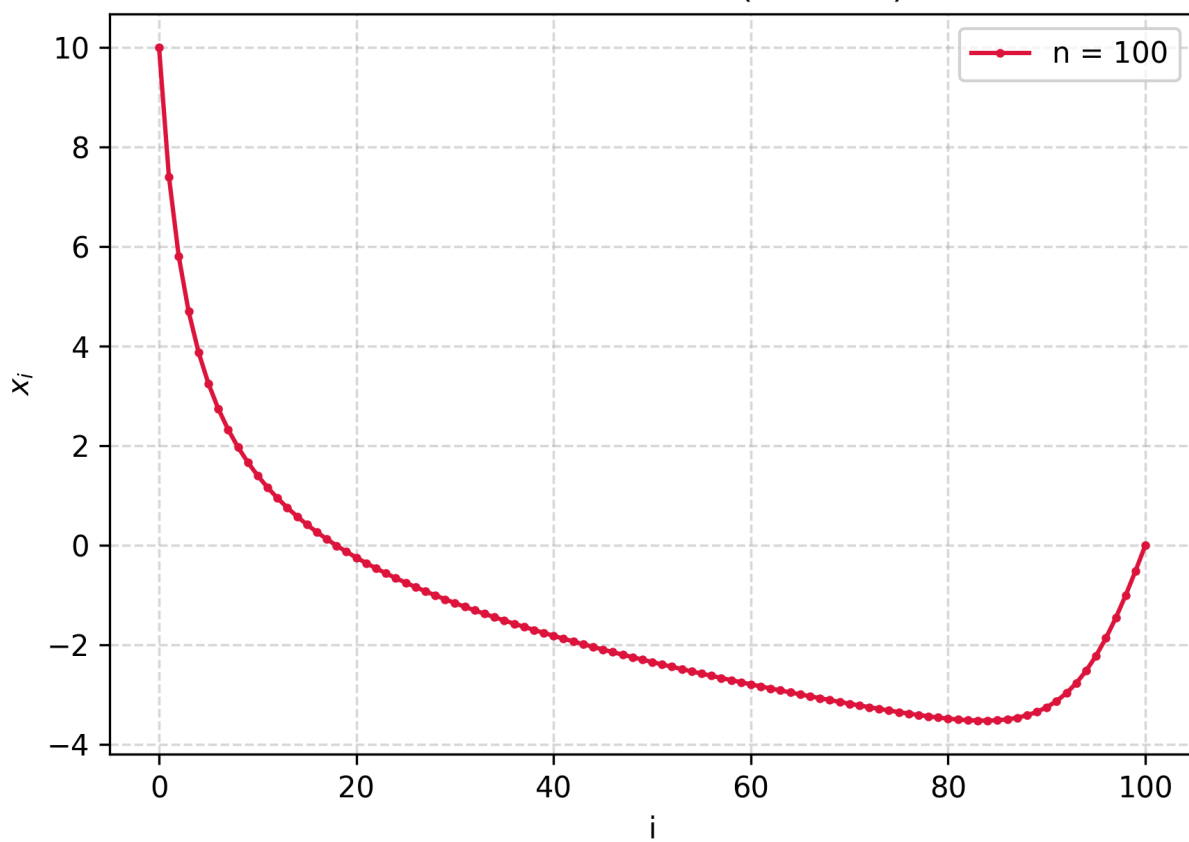
```

Диаграммы

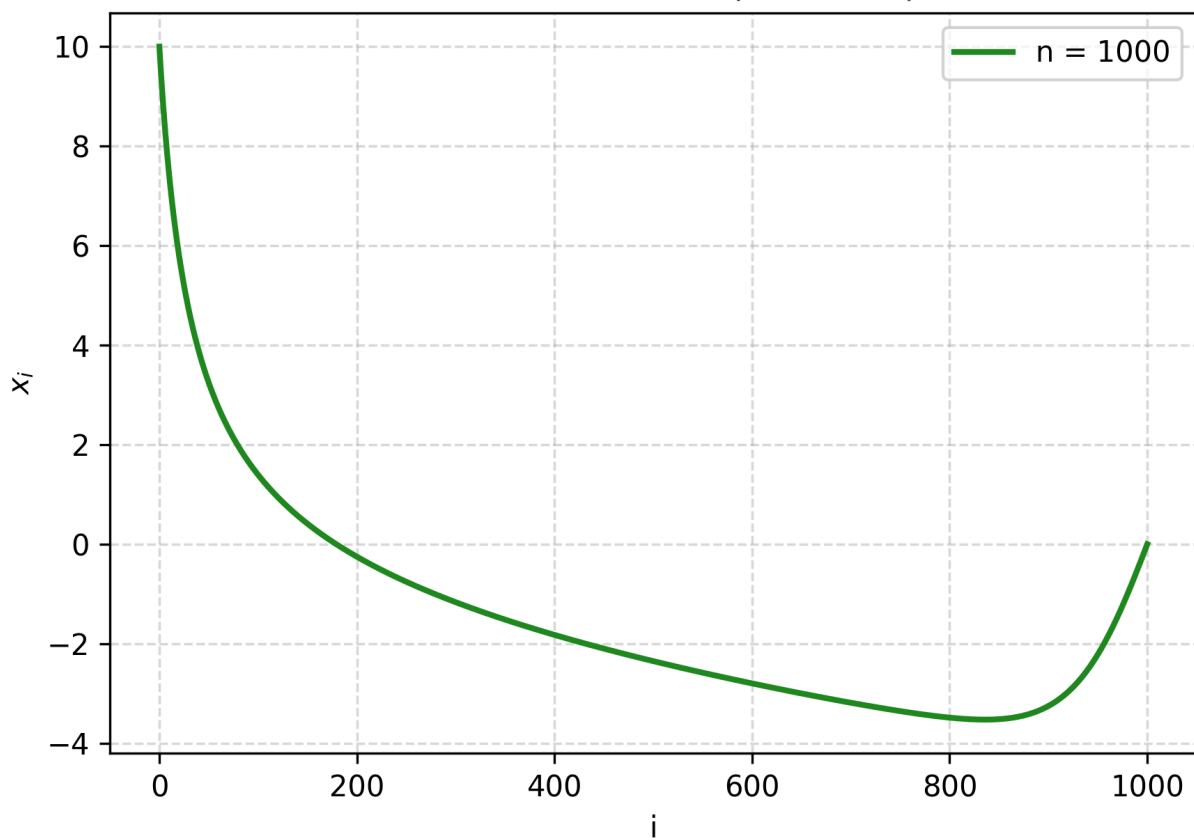
Получили следующие графики:



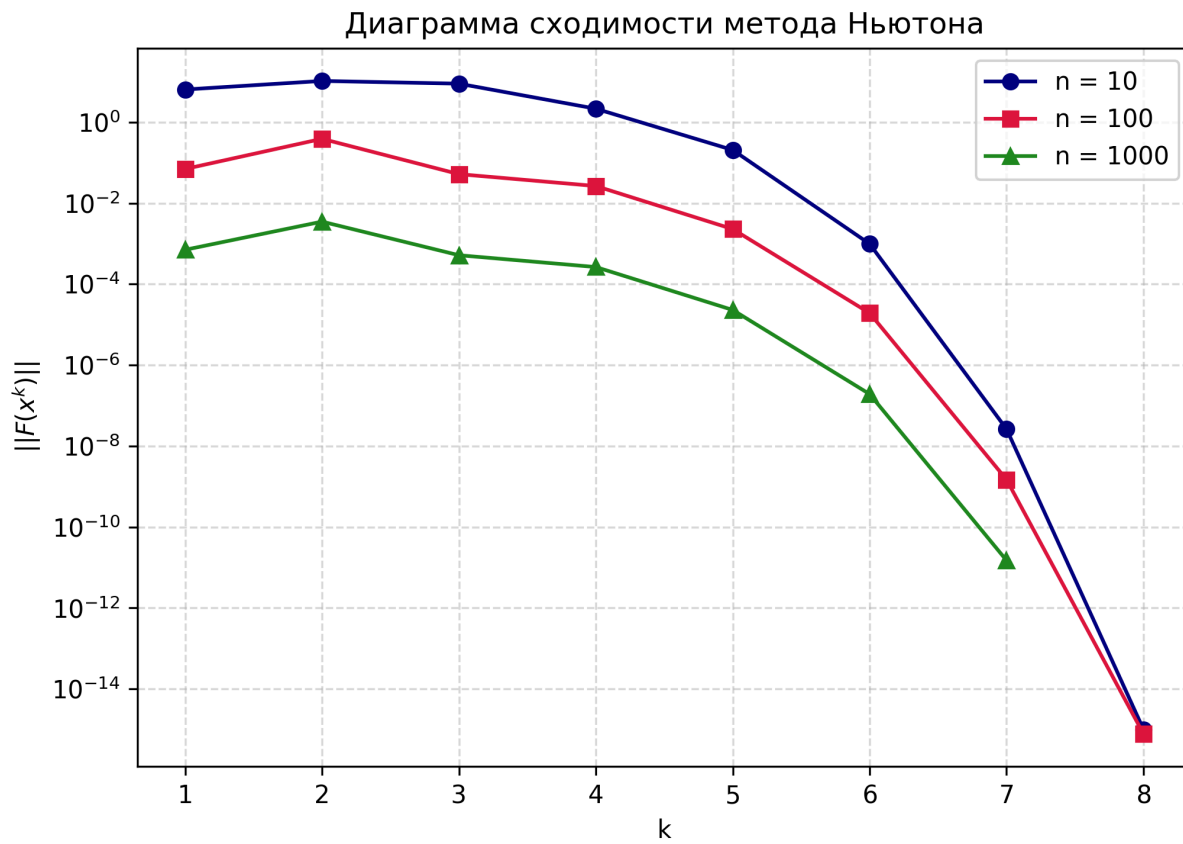
Решение системы ($n = 100$)



Решение системы ($n = 1000$)



Форма решения одинакова для всех n , условия на $x_0 = 10$, $x_n = 0$ выполняются.



Наблюдаемый второй порядок сходимости метода соответствует теоретическому. Решения сходятся одинаково быстро независимо от количества уравнений n .